ТЕХНИЧЕСКИ УНИВЕРСИТЕТ, СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВА РАБОТА

Дисциплина: Високопроизводителни компютърни системи

Тема:

Сравнителен симулационен анализ на архитектури с последователно и извънредно изпълнение чрез gem5

Изготвил:
Асен Пантелеев Попов
Фак. № 121223246
Група 38
III курс, КСИ
e-mail: `asepopov@tu-sofia.bg`

Ръководител:
доц. д-р инж. Ива Николова

София, 2026 г.

Съдържание

1 Увод **2**

2 Цел на проекта **2**

3 Теоретична основа **2**
3.1 Паралелизъм на ниво инструкции 2
3.2 In-order изпълнение 2
3.3 Out-of-order изпълнение 2
3.4 Метрики 3
3.5 Матрично умножение и невронни мрежи 3

4 Методология на експеримента **4**
4.1 Симулационна среда 4
4.2 Конфигурация на системата 4
4.3 Използвани натоварвания 5
4.4 Събрани статистики 6
4.5 Валидност на сравнението 6

5 Реализация **6**
5.1 Публично хранилище на проекта 7

6 Резултати **7**
6.1 Основна сравнителна таблица 7
6.2 Производни показатели 7
6.3 Графичен анализ 8
6.4 Количествени изводи 9

7 Обсъждане **9**

8 Ограничения и бъдеща работа **10**

9 Заключение **10**

Литература **10**

1 Увод

Съвременните високопроизводителни компютърни системи се стремят да повишават производителността не само чрез по-висока тактова честота, но и чрез по-ефективна микроархитектура. Един от основните архитектурни избори е дали инструкциите да се изпълняват строго в програмния ред или процесорът да може динамично да пренарежда изпълнението им. Този избор е особено важен при натоварвания, при които има разлика между изчислителна интензивност и латентност на паметта.

Настоящата курсова работа изследва чрез симулации в `gem5` поведението на две архитектури: `MinorCPU`, който реализира in-order изпълнение, и `O3CPU`, който реализира out-of-order изпълнение. Целта е да се анализира как архитектурният модел влияе върху IPC, CPI, общия брой цикли и кеш поведението при два типа натоварване: матрично умножение и pointer chasing.

2 Цел на проекта

Основната цел е да се извърши сравнителен анализ между in-order и out-of-order процесорна архитектура при еднакви системни условия. За да бъде сравнението коректно, и двете конфигурации използват една и съща ISA платформа, `X86`, еднаква честота от `3 GHz`, една и съща кеш йерархия с частни `L1I 32 KiB`, `L1D 32 KiB` и частен `L2 512 KiB`, както и една и съща памет `SingleChannelDDR3_1600`. Следователно единствената променлива в експеримента е типът процесорно ядро, `MinorCPU` като in-order модел и `O3CPU` като out-of-order модел.

3 Теоретична основа

3.1 Паралелизъм на ниво инструкции

Паралелизмът на ниво инструкции (*Instruction-Level Parallelism*, ILP) описва доколко различни инструкции могат да се изпълняват паралелно, без да нарушават коректността на програмата. Колкото повече независими инструкции има, толкова по-голям IPC може да постигне процесорът. Това е особено важно при изчислително-интензивни задачи, където има достатъчно операции, които могат да се подготвят и изпълняват едновременно.

3.2 In-order изпълнение

При in-order архитектурата инструкциите преминават през конвейера в реда, в който се срещат в програмата. Това опростява хардуера, но прави процесора по-чувствителен към stall-и. Когато дадена инструкция трябва да изчака резултат от паметта или от предходна зависимост, следващите инструкции също се забавят.

3.3 Out-of-order изпълнение

При out-of-order архитектурата процесорът може да избира готови за изпълнение инструкции, дори ако по-ранни инструкции все още чакат. Това се реализира чрез механизми като register renaming, instruction window, reorder buffer и dynamic scheduling. Благодарение на тях процесорът може да скрива част от латентността на паметта

и да използва по-добре наличните изпълнителни блокове. Предимството е по-висок IPC, а недостатъкът е по-сложен и по-скъп за симулация и реализация хардуер.

3.4 Метрики

В анализа се използват следните основни метрики:

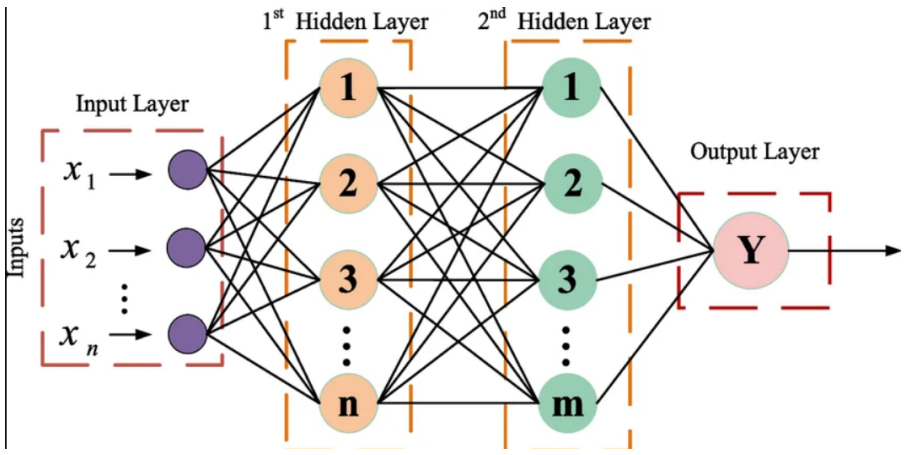
$$\mathrm{IPC} = \frac{\mathrm{Instructions}}{\mathrm{Cycles}} \tag{1}$$

$$\mathrm{CPI} = \frac{\mathrm{Cycles}}{\mathrm{Instructions}} \tag{2}$$

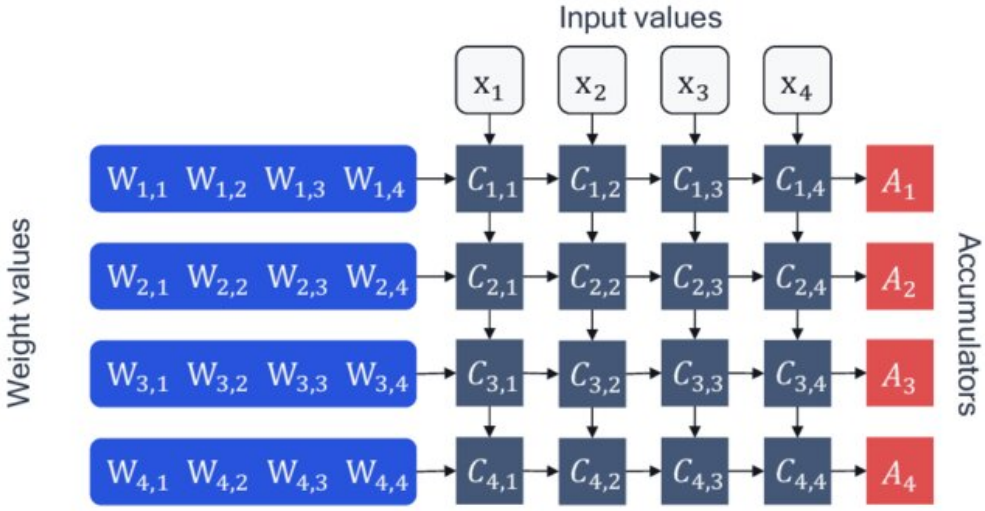
По-висок IPC и по-нисък CPI означават по-добро използване на процесора. Освен това се разглеждат: общият брой цикли, L1D miss rate и L2 miss rate. Тези показатели позволяват едновременно да се оцени изчислителната ефективност и влиянието на паметната йерархия върху поведението на двата процесорни модела.

3.5 Матрично умножение и невронни мрежи

Матричното умножение е фундаментална операция не само в класическите числени алгоритми, но и в съвременните системи с изкуствен интелект. При плътно свързан невронен слой изходният вектор се изчислява като $\mathbf{a} = \phi(W\mathbf{x} + \mathbf{b})$, където W е матрица от тегла, $\mathbf{x}$ е входен вектор, $\mathbf{b}$ е bias вектор, а ϕ е активационна функция. При обработка на множество наблюдения едновременно същата операция естествено се формулира като матрично умножение. Поради това производителността на операции от типа GEMM е пряко свързана с ефективността на inference и training натоварвания в невронните мрежи [5]. На практика значителна част от изчислителното натоварване при линейни слоеве, attention механизми и convolution lowering техники се свежда именно до такива матрични операции. Това означава, че наблюденията върху поведението на матричното умножение в рамките на настоящата работа имат пряка връзка с по-широк клас реални AI изчисления. В контекста на проекта benchmark-ът за матрично умножение е подходящ не само като типично изчислително-интензивно натоварване, но и като опростен представител на реални приложения.



Фигура 1: Примерна структура на multilayer perceptron, при която връзките между слоевете се реализират чрез матрични операции [6].

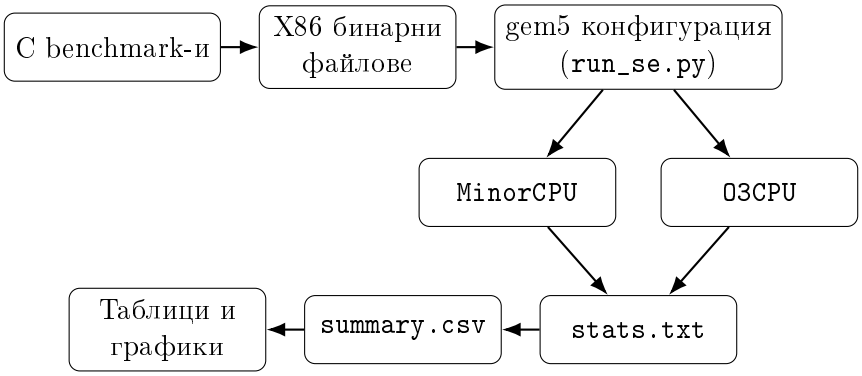


Фигура 2: Илюстрация на ролята на матричното умножение при изчисляването на изходите на невронен слой [5].

4 Методология на експеримента

4.1 Симулационна среда

Експериментите са изпълнени със симулатора `gem5`, компилиран като `build/X86/gem5.opt`. Използван е syscall-emulation режим, който позволява директно изпълнение на локални `X86` Linux бинарни файлове без пълна операционна система в симулацията. Този подход е достатъчен за целите на задачата, тъй като изследователският въпрос е фокусиран върху микроархитектурната разлика между `MinorCPU` и `O3CPU`, а не върху full-system поведение.



Фигура 3: Работен поток на експеримента от benchmark-ите до крайните резултати.

4.2 Конфигурация на системата

И за двата CPU модела е използвана една и съща системна конфигурация: едно ядро върху `X86Board`, тактова честота `3 GHz`, кеш йерархия с `L1I 32 KiB`, `L1D 32 KiB` и частен `L2 512 KiB`, както и `SingleChannelDDR3_1600` с `3 GiB` симулирана памет. Този избор гарантира, че отчетените разлики се дължат на микроархитектурния модел, а не на различна паметна или кеш конфигурация.

Параметър	Стойност
ISA / платформа	`X86Board`
Режим на симулация	syscall emulation (SE)
Брой ядра	1
Честота	`3 GHz`
L1 instruction cache	`32 KiB`
L1 data cache	`32 KiB`
L2 cache	`512 KiB`
Основна памет	`SingleChannelDDR3_1600`
Симулирана памет	`3 GiB`
Сравнявана променлива	`MinorCPU` срещу `O3CPU`
Натоварвания	`matrix_multiply`, `pointer_chase`

Таблица 1: Фиксирани параметри на експерименталната конфигурация

4.3 Използвани натоварвания

Използвани са два benchmark-а, създадени специално за проекта. Те представляват два различни режима на поведение: изчислително-интензивно натоварване с добра локалност и натоварване, ограничено от латентността на паметта и от зависимите адресации.

Матрично умножение. Това натоварване изпълнява класически тройно вложен цикъл за умножение на матрици. Характеризира се с добра локалност на данните и с висока изчислителна интензивност. Подобен тип операция е в основата и на плътните слоеве в невронните мрежи, поради което benchmark-ът има и приложна стойност извън чисто числения пример. Очакването е out-of-order архитектурата да извлече повече ILP и да постигне значително по-висок IPC.

```
for (long i = 0; i < n; ++i) {
    for (long j = 0; j < n; ++j) {
        int64_t sum = 0;

        for (long k = 0; k < n; ++k) {
            sum += (int64_t)a[(size_t)i * (size_t)n + (size_t)k] *
                   (int64_t)bt[(size_t)j * (size_t)n + (size_t)k];
        }

        c[(size_t)i * (size_t)n + (size_t)j] = (int32_t)sum;
    }
}
```

Listing 1: Откъс от benchmark-а за матрично умножение

Кодът показва класическия тройно вложен цикъл, при който във вътрешния цикъл се извършват многократни операции умножение и натрупване. Това създава благоприятни условия за използване на ILP и за ефективно натоварване на изпълнителните блокове.

Pointer chasing. Това натоварване изгражда псевдослучайна свързана структура в масив и след това обхожда елементите чрез последователност от зависими индирек-

тни адресации. Характеризира се с ниска локалност и със силна зависимост между стъпките, което ограничава ILP и увеличава вероятността от cache misses. Очакването е предимството на `O3CPU` да бъде осезаемо по-малко, тъй като дори агресивното пренареждане на инструкции не може да скрие напълно зависимите латентности на паметта.

```
for (long i = 0; i < nodes; ++i) {
    long next_i = (i + 1) % nodes;
    next[order[i]] = order[next_i];
}

index = order[0];

for (uint64_t step = 0; step < (uint64_t)nodes * (uint64_t)steps_per_node;
     ++step) {
    index = next[index];
    checksum += (uint64_t)index;
}
```

Listing 2: Откъс от benchmark-а за pointer chasing

Тук всяка следваща адресация зависи от стойността, заредена в предходната стъпка. Именно тази зависимост прави натоварването чувствително към латентността на паметта и силно ограничава възможностите за пренареждане на инструкции.

4.4 Събрани статистики

От `stats.txt` за всяко изпълнение са извлечени `simInsts`, `numCycles`, `ipc`, `cpi`, `l1d miss rate` и `l2 miss rate`. Тъй като броят симулирани инструкции е еднакъв в рамките на всяка двойка сравнявани натоварвания, метриките за цикли, IPC и CPI могат да бъдат използвани директно за коректна оценка на производителността.

4.5 Валидност на сравнението

Сравнението между двата процесорни модела е конструирано така, че да изолира единствено ефекта от микроархитектурата. За всяка двойка експерименти натоварването, ISA платформата, кеш конфигурацията, паметната система и честотата са еднакви. Освен това броят симулирани инструкции е идентичен в рамките на всяка двойка: при матричното умножение и двата модела изпълняват 29 077 226 инструкции, а при pointer chasing и двата модела изпълняват 12 431 387 инструкции. Следователно разликите в IPC, CPI и общия брой цикли могат да бъдат интерпретирани като реален ефект от in-order спрямо out-of-order изпълнение, а не като следствие от различна програма или различна системна конфигурация.

5 Реализация

За проекта беше изградена автоматизирана експериментална рамка. Създадени бяха два локални benchmark-а на `C`, shell скрипт за компилация на бинарните файлове, конфигурационен Python скрипт за `gem5` и автоматизиран launcher за четирите експеримента: `matrix_minor`, `matrix_o3`, `pointer_minor` и `pointer_o3`. Резултатите бяха

обобщени автоматично в `summary.csv`, а върху тях бяха генерирани таблици и графики за отчета. По този начин целият експериментален процес, от компилацията до извличането на метриките, беше възпроизводим и еднакъв за всички конфигурации.

5.1 Публично хранилище на проекта

Кодът на проекта е публикуван в GitHub хранилище [7]: https://github.com/Apsie09/Gem5-Microarchitecture-Analysis. В него са включени benchmark програмите на `C`, конфигурационният код за `gem5`, shell и Python скриптовете за изпълнение на експериментите и извличане на статистиките, генерираните обобщени резултати, както и LaTeX изходният код на настоящия отчет в директорията `report`. Това позволява експериментите и документацията да бъдат проследени, проверени и повторени независимо.

6 Резултати

6.1 Основна сравнителна таблица

Натоварване	CPU	Инструкции (M)	Цикли (M)	IPC	CPI	L1D miss rate	L2 miss rate
Matrix	Minor	29.08	65.32	0.445	2.246	0.000869	0.015299
Matrix	O3	29.08	11.05	2.631	0.380	0.003427	0.015169
Pointer	Minor	12.43	159.69	0.078	12.845	0.357979	0.463546
Pointer	O3	12.43	112.25	0.111	9.030	0.372311	0.477100

Таблица 2: Сравнение между `MinorCPU` и `O3CPU` при двата типа натоварване

Таблицата показва ясно, че за матричното умножение `O3CPU` постига драстично по-висока ефективност от `MinorCPU`, докато при pointer chasing предимството остава, но е по-умерено. По този начин още на таблично ниво се вижда разликата между compute-bound и memory-bound поведение.

6.2 Производни показатели

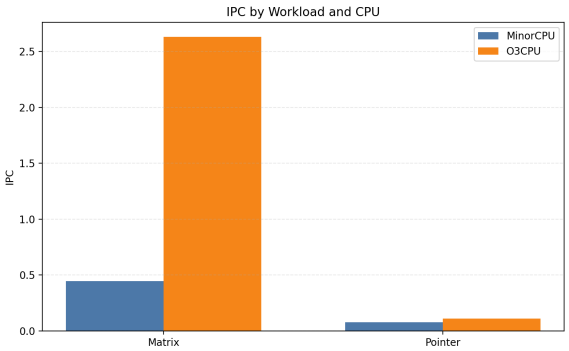
Натоварване	IPC speedup	Намаление на CPI	Намаление на циклите
Matrix	5.91×	83.08%	83.08%
Pointer	1.42×	29.70%	29.71%

Таблица 3: Обобщени производни показатели за `O3CPU` спрямо `MinorCPU`

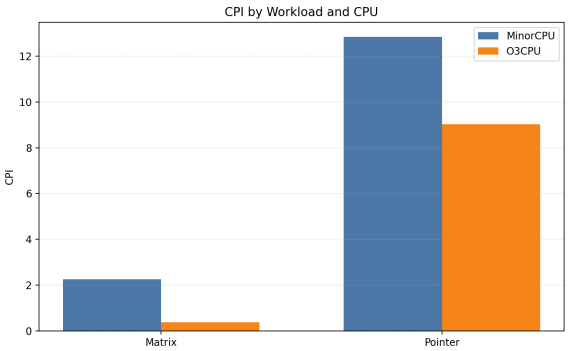
Тази таблица концентрира основния резултат на проекта в кратка форма. При матричното умножение out-of-order моделът носи много голямо ускорение, докато при pointer chasing подобрението е осезаемо, но ограничено от зависимостите между адресациите и от по-високата чувствителност към латентността на паметта.

6.3 Графичен анализ

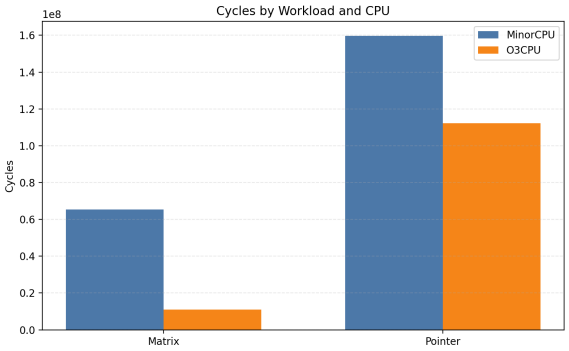
Графиките потвърждават изводите от табличния анализ и позволяват по-бързо визуално сравнение между двата модела. Най-ясно се откроява, че при матричното умножение разликата между `MinorCPU` и `O3CPU` е значително по-голяма, отколкото при pointer chasing, докато кеш miss rate показва, че по-високата производителност на `O3CPU` не произтича непременно от по-добра локалност, а от по-ефективно използване на наличния ILP и по-добро скриване на латентността.



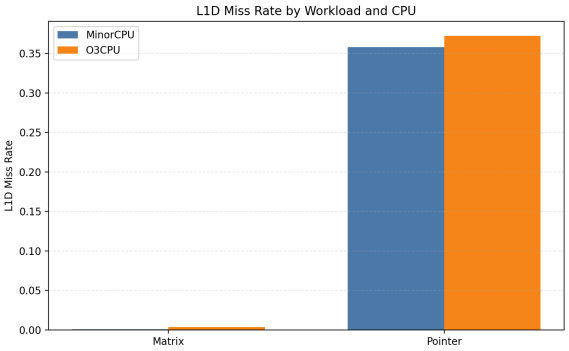
Фигура 4: IPC по натоварване и тип процесор.



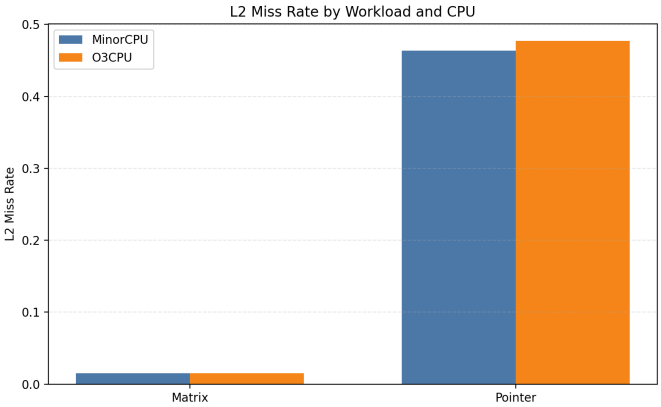
Фигура 5: CPI по натоварване и тип процесор.



Фигура 6: Общ брой цикли по натоварване и тип процесор.



Фигура 7: L1D miss rate по натоварване и тип процесор.



Фигура 8: L2 miss rate по натоварване и тип процесор.

Визуално представяне на графиките показва, че IPC и броят цикли реагират много по-силно на промяната в микроархитектурата при compute-bound натоварването, отколкото при memory-bound натоварването. От друга страна, L1D и L2 miss rate остават високи при pointer chasing и за двата процесорни модела, което подкрепя извода, че тук ограничението идва основно от паметната система.

6.4 Количествени изводи

За матричното умножение `O3CPU` постига приблизително 5.91× по-висок IPC спрямо `MinorCPU`. Едновременно с това CPI намалява с около 83.1%, а общият брой цикли също намалява с приблизително 83.1%. Това е силно изразено предимство на out-of-order архитектурата при compute-bound натоварване.

За pointer chasing `O3CPU` постига само около 1.42× по-висок IPC. CPI намалява с приблизително 29.7%, а общият брой цикли намалява с приблизително 29.7%. Предимството е реално, но е значително по-малко, което е очаквано при memory-bound задача.

7 Обсъждане

Получените резултати потвърждават основната хипотеза на проекта и показват ясно, че ефектът от out-of-order изпълнението зависи от характера на натоварването. При матричното умножение разликата между `MinorCPU` и `O3CPU` е много голяма, тъй като натоварването има добра локалност и съдържа достатъчно независими операции. Това позволява на `O3CPU` да използва по-ефективно изпълнителните блокове и да повиши значително IPC, като едновременно с това намали CPI и общия брой цикли.

При pointer chasing ситуацията е качествено различна. Тук всяка следваща адресация зависи от резултата от предходната, което силно ограничава ILP. Освен това и при двата CPU модела L1D и L2 miss rate остават високи. Следователно доминиращият фактор е латентността на паметта, а не способността на процесора да пренарежда независими инструкции. В такава среда `O3CPU` продължава да има предимство, но то е значително по-малко, отколкото при матричното умножение.

Особено показателен е фактът, че `O3CPU` не постига непременно по-нисък miss rate. При матричния benchmark стойността на L1D miss rate за `O3CPU` дори е малко по-висока от тази за `MinorCPU`, но въпреки това `O3CPU` е многократно по-бърз по отношение на симулираните цикли и IPC. Това показва, че основното предимство на out-of-order изпълнението не е задължително подобряване на локалността, а по-ефективно скриване на латентността и по-добро използване на готовите за изпълнение инструкции.

Следва да се разграничават и две различни понятия за производителност: симулираната производителност на моделируемия процесор и реалното време за изпълнение на симулацията върху хост машината. Например при pointer chasing `O3CPU` е по-добър по симулирани цикли, но изисква повече host време. Това не е противоречие, а естествен резултат от факта, че `O3CPU` е по-сложен микроархитектурен модел и съответно е по-скъп за симулация.

8 Ограничения и бъдеща работа

Настоящият експеримент използва само два benchmark-а и едноядрена конфигурация. Това е достатъчно за ясен първоначален сравнителен анализ, но не покрива всички възможни класове натоварвания. Освен това е използван syscall-emulation режим, а не full-system симулация. Този избор е оправдан с оглед на поставената задача, но изключва влиянието на операционната система, прекъсванията и по-сложните многопрограмни сценарии.

Като естествено продължение на работата могат да се разгледат по-голям набор от benchmark-и, вариране на кеш размери и латентности, сравнение при други ISA платформи и многоядрени конфигурации. Полезно направление е и включването на допълнителни натоварвания, свързани с линейна алгебра и AI inference, така че връзката между микроархитектурния модел и реалните приложения да бъде анализирана още по-непосредствено. Практически ценно разширение би било и по-тясното интегриране на автоматичното извличане на статистики с LaTeX workflow, така че таблиците и графиките да се обновяват автоматично при всяко ново изпълнение на експериментите.

9 Заключение

В настоящата курсова работа беше реализиран и изпълнен сравнителен симулационен анализ на `MinorCPU` и `O3CPU` чрез `gem5`. При еднакви кеш, памет и честота беше показано, че out-of-order архитектурата носи голямо предимство при изчислително-интензивно натоварване и по-ограничено, но все пак осезаемо предимство при натоварване, доминирано от латентността на паметта.

Най-силно изразеният резултат е при матричното умножение, където `O3CPU` постига приблизително 5.9× по-висок IPC и около 83% по-малко цикли. При pointer chasing предимството остава, но е значително по-слабо: приблизително 1.42× по-висок IPC и около 30% по-малко цикли. Следователно основният извод е, че ефектът от out-of-order изпълнението зависи силно от характера на натоварването и е най-голям тогава, когато съществува достатъчен ILP и когато паметната латентност не е доминиращият ограничител.

Литература

[1] The gem5 Developers. gem5 Documentation. Available at: https://www.gem5.org/documentation/

[2] Hennessy, J. L., Patterson, D. A. *Computer Architecture: A Quantitative Approach.* Morgan Kaufmann.

[3] The gem5 Developers. Minor CPU Model Documentation. Available at: https://www.gem5.org/documentation/general_docs/cpu_models/minor_cpu

[4] The gem5 Developers. O3CPU Model Documentation. Available at: https://www.gem5.org/documentation/general_docs/cpu_models/O3CPU

[5] S. Siddegowda, M. Fournarakis, M. Nagel, T. Blankevoort, C. Patel, A. Khobare. Neural Network Quantization with AI Model Efficiency Toolkit. arXiv:2201.08442, 2022.

[6] L. Zhang, B. Wang, Y. Shen, Y. Nie. An online soft sensor method for biochemical reaction process based on JS-ISSA-XGBoost. *BMC Biotechnology*, 23:49, 2023. Available at: https://doi.org/10.1186/s12896-023-00816-3

[7] A. Popov. Gem5 Microarchitecture Analysis. GitHub repository. Available at: https://github.com/Apsie09/Gem5-Microarchitecture-Analysis